

Assignment Code: DA-AG-015

Boosting Techniques | Assignment

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 200

Question 1: What is Boosting in Machine Learning? Explain how it improves weak learners.

Answer:

- Boosting is an ensemble technique that combines multiple weak learners (usually decision trees) to create a strong model. Each new model focuses on correcting the errors of the previous one.

↳ It improves performance by:

- Giving more importance to misclassified points
- Sequential learning
- Reducing bias and variance

Question 2: What is the difference between AdaBoost and Gradient Boosting in terms of how models are trained?

Answer:

Feature	AdaBoost	Gradient Boosting
-----	-----	-----
Training	Sequential	Sequential
Focus	Misclassified points	Error minimization (loss function)
Weight update	Changes data weights	Uses gradient descent
Complexity	Simple	More complex

Question 3: How does regularization help in XGBoost?

Answer:

- Regularization prevents overfitting by penalizing complex models.

☞ XGBoost uses:

- L1 regularization (Lasso)
- L2 regularization (Ridge)

This improves:

- Generalization
- Model stability

Question 4: Why is CatBoost considered efficient for handling categorical data?

Answer:

- CatBoost handles categorical data internally without encoding.

☞ Advantages:

- No need for one-hot encoding
- Uses ordered boosting

- Reduces overfitting
- Faster and more accurate

Question 5: What are some real-world applications where boosting techniques are preferred over bagging methods?

Answer:

- Boosting is preferred when high accuracy is needed:
- Fraud detection
- Loan default prediction
- Medical diagnosis
- Recommendation systems
- Customer churn prediction

Question 6: Write a Python program to:

Datasets:

- Use `sklearn.datasets.load_breast_cancer()` for classification tasks.
- Use `sklearn.datasets.fetch_california_housing()` for regression tasks.
- Train an AdaBoost Classifier on the Breast Cancer dataset
- Print the model accuracy

(Include your Python code and output in the code box below.)

Answer:**Code:**

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import AdaBoostClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target,
test_size=0.3)

model = AdaBoostClassifier(n_estimators=50)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Question 7: Write a Python program to:

- Train a Gradient Boosting Regressor on the California Housing dataset
- Evaluate performance using R-squared score

(Include your Python code and output in the code box below.)

Answer:

```
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

data = fetch_california_housing()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target)

model = GradientBoostingRegressor()
model.fit(X_train, y_train)

pred = model.predict(X_test)
print("R2 Score:", r2_score(y_test, pred))
```

Question 8: Write a Python program to:

- Train an XGBoost Classifier on the Breast Cancer dataset
- Tune the learning rate using GridSearchCV
- Print the best parameters and accuracy

(Include your Python code and output in the code box below.)

Answer:

Code:

```
from xgboost import XGBClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target)

param_grid = {
    'learning_rate': [0.01, 0.1, 0.2]
}

model = XGBClassifier()

grid = GridSearchCV(model, param_grid, cv=3)
grid.fit(X_train, y_train)

print("Best Params:", grid.best_params_)

pred = grid.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
```

Question 9: Write a Python program to:

- Train a CatBoost Classifier
- Plot the confusion matrix using **seaborn**

(Include your Python code and output in the code box below.)

Answer:

Code:

```
from catboost import CatBoostClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target)

model = CatBoostClassifier(verbose=0)
model.fit(X_train, y_train)

pred = model.predict(X_test)

cm = confusion_matrix(y_test, pred)

sns.heatmap(cm, annot=True, fmt='d')
plt.show()
```

Question 10: You're working for a FinTech company trying to predict loan default using customer demographics and transaction behavior.

The dataset is imbalanced, contains missing values, and has both numeric and categorical features.

Describe your step-by-step data science pipeline using boosting techniques:

- Data preprocessing & handling missing/categorical values
- Choice between AdaBoost, XGBoost, or CatBoost
- Hyperparameter tuning strategy
- Evaluation metrics you'd choose and why
- How the business would benefit from your model

(Include your Python code and output in the code box below.)

Answer:

✓ 1. Data Preprocessing

Handle missing values (mean/median)

Encode categorical features (or use CatBoost)

Normalize numerical features

Handle imbalance using SMOTE or class weights

✓ 2. Model Choice

CatBoost → best for categorical + missing values

XGBoost → best for performance and tuning

AdaBoost → simple baseline

👉 Recommended: CatBoost

✓ 3. Hyperparameter Tuning

Use GridSearchCV / RandomizedSearch:

learning_rate

depth

n_estimators

✓ 4. Evaluation Metrics

Use:

Accuracy (basic)

Precision (reduce false positives)

Recall (important for defaults)

F1-score (balance)

ROC-AUC (best for imbalance)

✓ 5. Business Benefits

Reduce loan defaults

Improve risk assessment

Increase profit

Better customer targeting